

Low-Level Design (LLD)

Project: RAG-Based Customer Support Assistant

1. Module-Level Design

- Ingestion Module: PyPDFLoader → RecursiveCharacterTextSplitter (chunk_size=512, overlap=50).
- Embedding & Storage Module: Generates 384-dimensional vectors using BAAI/bge-small-en-v1.5 → Stores in ChromaDB.
- Retrieval Module: Uses Maximal Marginal Relevance (MMR) to fetch top k=3 chunks.
- Graph Orchestration Module: LangGraph StateGraph managing node transitions.
- LLM Module: Groq ChatGroq (Llama-3.1-8b) invoked with strict JSON output parsing.

2. Data Structures (State & Schemas)

LangGraph State Object (Python TypedDict)

This object is passed between all nodes during execution:

```
Python ⌵ 📋  
  
class GraphState(TypedDict):  
    query: str          # User's original input  
    intent: str         # "support", "escalate", or "greeting"  
    context: List[Document] # Retrieved chunks from ChromaDB  
    generation: str     # The LLM's drafted answer  
    confidence: float   # 0.0 to 1.0 score assigned by LLM  
    human_override: str # Text injected by human agent (if HITL triggered)
```

3. Workflow Design (LangGraph)

Nodes (Functions executing logic):

1. route_query(state): Analyzes intent, updates state["intent"].
2. retrieve(state): Queries ChromaDB, updates state["context"].
3. generate(state): Drafts answer and confidence score. Updates state["generation"] & state["confidence"].
4. hitl_node(state): Dummy node serving as the execution breakpoint.

Edges (Flow Control):

- START → route_query
- Conditional Edge (Router): * If intent == "support" → retrieve
 - If intent == "escalate" → hitl_node
- retrieve → generate
- Conditional Edge (Evaluator):
 - If confidence >= 0.8 → END
 - If confidence < 0.8 → hitl_node
- hitl_node (Wait for input) → Updates state → END

4. Conditional Routing Logic

Executed by a lightweight LLM call utilizing JSON mode:

- Escalate Criteria: Query contains keywords (agent, manager, refund, angry, sue) OR sentiment is highly negative.
- Support Criteria: Query asks "how to", "what is", or requests factual information.
- Low Confidence Trigger: LLM evaluates its drafted answer against the retrieved context. If hallucination risk is high (score < 0.8) or context is empty, trigger escalation.

5. HITL Design (Human-in-the-Loop)

1. Trigger: LangGraph interrupt_before=["hitl_node"] halts the graph.
2. State Persistence: The current GraphState is saved to memory (MemorySaver checkpoint).
3. Human Action: An agent reviews state["query"] and state["generation"] (if any), then submits a human_override string via the UI.
4. Resumption: Graph resumes execution, replacing state["generation"] with state["human_override"] before routing to END.

6. API / Interface Design

Input Payload (POST /chat)

JSON

```
{  
  "user_id": "12345",  
  "thread_id": "thread_abc",  
  "query": "How do I reset my password?"
```

```
}
```

Output Payload (200 OK)

JSON

```
{  
  "status": "success",  
  "response": "To reset your password, go to settings...",  
  "escalated": false,  
  "sources": ["doc_page_5.pdf", "doc_page_8.pdf"]  
}
```

7. Error Handling

- No Relevant Chunks Found: retrieve node returns empty list. generate node detects empty context → Auto-triggers HITL node.
- LLM API Timeout/Failure: Caught via Python try/except. Retries 3 times using LangChain's `.with_retry()`. If it still fails, returns a static fallback string: *"System unavailable. Connecting you to an agent."* and routes to HITL.